Bash basics

Bash basics

- The **shell** is how you talk to the operating system in text.

Bash basics

- The **shell** is how you talk to the operating system in text.
- **Bash** is the default shell on most Linux systems and inside the class container.

Bash basics

- The **shell** is how you talk to the operating system in text.
- **Bash** is the default shell on most Linux systems and inside the class container.
- Learning a handful of commands makes you faster in any environment: laptop, server, or Docker.

Navigation

- **pwd** — print the current working directory.

Navigation

- **`pwd`** — print the current working directory.
- **`ls`** — list directory contents.

Navigation

- **`pwd`** — print the current working directory.
- **`ls`** — list directory contents.
- **`cd`** — change directory.

Navigation

- **`pwd`** — print the current working directory.
- **`ls`** — list directory contents.
- **`cd`** — change directory.

Navigation

- **pwd** — print the current working directory.
- **ls** — list directory contents.
- **cd** — change directory.

```
pwd
ls -l           # long format
ls -a           # include hidden files
cd /workspace   # absolute path
cd ..           # up one level
cd ~            # home directory
cd -            # previous directory
```

File management

- **`touch`** — create an empty file (or update its timestamp).

File management

- **`touch`** — create an empty file (or update its timestamp).
- **`mkdir`** — create a directory.

File management

- **`touch`** — create an empty file (or update its timestamp).
- **`mkdir`** — create a directory.
- **`rm`** — remove files or directories (**`-r`** for recursive).

File management

- **`touch`** — create an empty file (or update its timestamp).
- **`mkdir`** — create a directory.
- **`rm`** — remove files or directories (**`-r`** for recursive).

File management

- **`touch`** — create an empty file (or update its timestamp).
- **`mkdir`** — create a directory.
- **`rm`** — remove files or directories (**`-r`** for recursive).

```
touch newfile.txt
mkdir results
rm file.txt
rm -r old_results/
```

File management

- **`touch`** — create an empty file (or update its timestamp).
- **`mkdir`** — create a directory.
- **`rm`** — remove files or directories (**`-r`** for recursive).

```
touch newfile.txt
mkdir results
rm file.txt
rm -r old_results/
```

- Deletion is **immediate** and **permanent**. There is no trash can.

Copying and moving

- **`cp`** — copy files or directories (**`-r`** for recursive).

Copying and moving

- **`cp`** — copy files or directories (**`-r`** for recursive).
- **`mv`** — move or rename.

Copying and moving

- **`cp`** — copy files or directories (**`-r`** for recursive).
- **`mv`** — move or rename.

Copying and moving

- **cp** — copy files or directories (**-r** for recursive).
- **mv** — move or rename.

```
cp source.txt backup.txt
cp -r data/ data_backup/
mv draft.txt final.txt
mv report.pdf /workspace/output/
```

Copying and moving

- **`cp`** — copy files or directories (**`-r`** for recursive).
- **`mv`** — move or rename.

```
cp source.txt backup.txt
cp -r data/ data_backup/
mv draft.txt final.txt
mv report.pdf /workspace/output/
```

- Both commands **overwrite** silently. Use **`-i`** to confirm first.

Wildcards (globbing)

- The shell expands **patterns** into matching filenames **before** the command runs.

Wildcards (globbing)

- The shell expands **patterns** into matching filenames **before** the command runs.
- If nothing matches, bash leaves the pattern as literal text (which can confuse the next command).

Wildcards (globbing)

- The shell expands **patterns** into matching filenames **before** the command runs.
- If nothing matches, bash leaves the pattern as literal text (which can confuse the next command).

Wildcards (globbing)

- The shell expands **patterns** into matching filenames **before** the command runs.
- If nothing matches, bash leaves the pattern as literal text (which can confuse the next command).

```
ls *.txt                # every name ending in .txt in this directory
ls data/*.csv             # all .csv files inside data/
rm fig_?.png              # fig_1.png, fig_A.png — one character where ? is
cp notes[12].md backup/ # notes1.md and notes2.md only
```

Wildcards (globbing)

- The shell expands **patterns** into matching filenames **before** the command runs.
- If nothing matches, bash leaves the pattern as literal text (which can confuse the next command).

```
ls *.txt                # every name ending in .txt in this directory
ls data/*.csv             # all .csv files inside data/
rm fig_?.png              # fig_1.png, fig_A.png — one character where ? is
cp notes[12].md backup/ # notes1.md and notes2.md only
```

- `*` — any sequence of characters (including none).

Wildcards (globbing)

- The shell expands **patterns** into matching filenames **before** the command runs.
- If nothing matches, bash leaves the pattern as literal text (which can confuse the next command).

```
ls *.txt                # every name ending in .txt in this directory
ls data/*.csv             # all .csv files inside data/
rm fig_?.png              # fig_1.png, fig_A.png — one character where ? is
cp notes[12].md backup/ # notes1.md and notes2.md only
```

- `*` — any sequence of characters (including none).
- `?` — exactly one character.

Wildcards (globbing)

- The shell expands **patterns** into matching filenames **before** the command runs.
- If nothing matches, bash leaves the pattern as literal text (which can confuse the next command).

```
ls *.txt                # every name ending in .txt in this directory
ls data/*.csv             # all .csv files inside data/
rm fig_?.png              # fig_1.png, fig_A.png — one character where ? is
cp notes[12].md backup/ # notes1.md and notes2.md only
```

- `*` — any sequence of characters (including none).
- `?` — exactly one character.
- `[...]` — one character from the set or range (`[0-9]`, `[a-z]`).

Wildcards (globbing)

- The shell expands **patterns** into matching filenames **before** the command runs.
- If nothing matches, bash leaves the pattern as literal text (which can confuse the next command).

```
ls *.txt                # every name ending in .txt in this directory
ls data/*.csv             # all .csv files inside data/
rm fig_?.png              # fig_1.png, fig_A.png — one character where ? is
cp notes[12].md backup/ # notes1.md and notes2.md only
```

- `*` — any sequence of characters (including none).
- `?` — exactly one character.
- `[...]` — one character from the set or range (`[0-9]`, `[a-z]`).
- `{a,b,c}` — **brace expansion**: bash builds several words (`file.{txt,md}` → `file.txt file.md`). Not the same as `*`, but handy with `cp`, `mv`, and `mkdir`.

File permissions

```
ls -l
# -rwxr-xr-- 1 user group 1234 Jan 1 12:34 file.txt
```

File permissions

```
ls -l
# -rwxr-xr-- 1 user group 1234 Jan 1 12:34 file.txt
```

- **10-character string:** file type + owner / group / others, each `rwx`.

File permissions

```
ls -l
# -rwxr-xr-- 1 user group 1234 Jan 1 12:34 file.txt
```

- **10-character string:** file type + owner / group / others, each **rwx**.
- **chmod** changes permissions; **chown** changes ownership.

File permissions

```
ls -l
# -rwxr-xr-- 1 user group 1234 Jan 1 12:34 file.txt
```

- **10-character string:** file type + owner / group / others, each **rwx**.
- **chmod** changes permissions; **chown** changes ownership.

File permissions

```
ls -l
# -rwxr-xr-- 1 user group 1234 Jan 1 12:34 file.txt
```

- **10-character string:** file type + owner / group / others, each **rwx**.
- **chmod** changes permissions; **chown** changes ownership.

```
chmod 755 file.txt    # rwxr-xr-x
chmod +x script.sh    # add execute permission
```

File permissions

```
ls -l
# -rwxr-xr-- 1 user group 1234 Jan 1 12:34 file.txt
```

- 10-character string: file type + owner / group / others, each **rwx**.
- **chmod** changes permissions; **chown** changes ownership.

```
chmod 755 file.txt    # rwxr-xr-x
chmod +x script.sh    # add execute permission
```

- New scripts cannot be run until you `chmod +x` them.

Viewing and editing text

- `cat` — print file contents.

Viewing and editing text

- `cat` — print file contents.
- `less` — scroll through a file (`q` to quit).

Viewing and editing text

- `cat` — print file contents.
- `less` — scroll through a file (`q` to quit).
- `nano` — simple terminal editor.

Viewing and editing text

- `cat` — print file contents.
- `less` — scroll through a file (`q` to quit).
- `nano` — simple terminal editor.
- `echo` — print text; combine with redirection to write files.

Viewing and editing text

- `cat` — print file contents.
- `less` — scroll through a file (`q` to quit).
- `nano` — simple terminal editor.
- `echo` — print text; combine with redirection to write files.

Viewing and editing text

- **cat** — print file contents.
- **less** — scroll through a file (**q** to quit).
- **nano** — simple terminal editor.
- **echo** — print text; combine with redirection to write files.

```
cat file.txt
less long_output.txt
nano notes.md
echo "hello" > greet.txt      # overwrite
echo "world" >> greet.txt     # append
```

Pipes

- The pipe | sends the output of one command into the input of the next.

Pipes

- The pipe | sends the output of one command into the input of the next.

Pipes

- The pipe `|` sends the output of one command into the input of the next.

```
ls -l | less                    # browse long listing
grep "error" log.txt | wc -l    # count matching lines
cat data.csv | sort | uniq      # sort and deduplicate
```

Redirection

- `>` overwrites a file with command output.

Redirection

- `>` overwrites a file with command output.
- `>>` appends to the file.

Redirection

- `>` overwrites a file with command output.
- `>>` appends to the file.

Redirection

- `>` overwrites a file with command output.
- `>>` appends to the file.

```
echo "Hello" > output.txt
echo "World" >> output.txt
ls -l | grep ".py" > python_files.txt
```

Summary

- Navigate: `pwd`, `ls`, `cd`.

Summary

- Navigate: `pwd`, `ls`, `cd`.
- Manage files: `touch`, `mkdir`, `rm`, `cp`, `mv`; patterns with `*`, `?`, `[...]`, `{a,b}`.

Summary

- Navigate: `pwd`, `ls`, `cd`.
- Manage files: `touch`, `mkdir`, `rm`, `cp`, `mv`; patterns with `*`, `?`, `[...]`, `{a,b}`.
- View/edit: `cat`, `less`, `nano`, `echo`.

Summary

- **Navigate:** `pwd`, `ls`, `cd`.
- **Manage files:** `touch`, `mkdir`, `rm`, `cp`, `mv`; patterns with `*`, `?`, `[...]`, `{a,b}`.
- **View/edit:** `cat`, `less`, `nano`, `echo`.
- **Compose:** pipes (`|`) and redirection (`>`, `>>`).

Summary

- Navigate: `pwd`, `ls`, `cd`.
- Manage files: `touch`, `mkdir`, `rm`, `cp`, `mv`; patterns with `*`, `?`, `[...]`, `{a,b}`.
- View/edit: `cat`, `less`, `nano`, `echo`.
- Compose: pipes (`|`) and redirection (`>`, `>>`).
- Permissions: `chmod`, `chown`, `ls -l`.